

Lecture 1b

Basic Python

About Strings

A **string** is any sequence of characters. We usually use them to store words, sentences or even entire paragraphs.

The data type of a string is **str**.

Creating a String

String can begin and end with either **single quotes** or **double quotes**.

```
spam = "That is Alice."
```

or

```
spam = 'That is Alice.'
```

String can contain single quote and we can type :

```
spam = "That is Alice's cat."
```

But not

```
spam = 'That is Alice's cat.' (This will  
not work !)
```

Escape Characters

Escape Characters	Prints as	Example	Output String
\'	Single quote	'Alice\'s cat.'	Alice's cat.
\"	Double quote	"Type the \"Double Quote\" here."	Type the "Double Quote" here.
\t	Tab	print('Class \tName \tTel')	Class Name Tel
\n	Newline (line break)	print('Class \nName \nTel')	Class Name Tel
\\	Backslash	'Type the backslash \\ here.'	Type the backslash \ here.

spam = 'That is Alice\'s cat.'
(This will work now!)

Comments

The hash character (#) marks the beginning of a comment within a Python code.

Eg .

```
# Comment for Python program.
```

Within Jupyter, you can use `Ctrl + /` to toggle between commented and uncommented.
More shortcut keys at the end of this pdf.

Multi-line Comment

The **triple quotes**, either single or double, can be used to mark the beginning and end of a multi-line comment within a Python code.

Eg .

```
'''This is a test Python program.  
Use this as a template.  
Do not remove any line.'''
```

The *in* and *not in* Operators with Strings

The operators *in* and *not in* will evaluate to a Boolean *True* or *False*.

Eg .

```
'Hello' in 'Hello World'
```

```
True
```

```
'HELLO' in 'Hello World'
```

```
False
```

The + and * Operators with Strings

The arithmetic operators + and * can be used for string **concatenation** and **replication**.

Eg .

```
'Hello ' + 'World'
```

```
'Hello World'
```

```
'HELLO ' * 3
```

```
'HELLO HELLO HELLO '
```

The >, <, ==, >=, and <= Operators with Strings

The inequality operators can be used to compare strings. They will evaluate to a Boolean `True` or `False`.

Eg .

```
'Ten' > 'One'
```

```
True
```

```
'FIVE' == 5
```

```
False
```

Some String Operations

```
s = 'ba'
```

```
t = 'ck'
```

```
s + t
```

```
'back'
```

```
t = s + 'na' * 2
```

```
t
```

```
'banana'
```

```
'z' in t
```

```
False
```

```
'bananb' > t
```

```
False
```

```
'banan' <= t
```

```
True
```

```
'c' < t
```

```
False
```

Useful String Methods

```
len('Hello World')  
11
```

There are other useful methods but we will not cover them in our lesson. Do explore them in your own time.

Eg : `upper()`, `lower()`, `startswith()`, `endswith()`, `strip()`, `lstrip()`, `rstrip()`, `split()`, `join()`, `format()`, `isalpha()`, `isalnum()`, `isdigit()`, `isdecimal()`, `isspace()`, `isupper()`, `islower()`, `istitle()`, `index()`, `encode()`, `center()`, `ljust()`, `rjust()`

1. Index Operator on String



Index a String

Example :

```
s = 'abcd'
```

```
s[0]
```

```
'a'
```

```
s[2]
```

```
'c'
```

First character is indexed with 0.

2. String Slicing



3. String Slicing with Step



String Slicing

`s[start:stop:step]`

 non-inclusive

```
s = 'abcdef'
```

```
s[0:2]
```

```
'ab'
```

```
s[1:2]
```

```
'b'
```

```
s[:2]
```

```
'ab'
```

```
s[1:5:3]
```

```
'be'
```

```
s[::-2]
```

```
'ace'
```

Important : Slicing always returns a new string.

One way to consider string slicing

- For start and stop values, think of it as placing a cursor to the left of the position

```
s = 'abcdefgh'
```

```
s[1:6]
```

A diagram illustrating string slicing on the string 'abcdefgh'. The string is displayed in white text on a dark blue rectangular background. Two vertical yellow lines are positioned to the left of the characters 'a' and 'f'. The first line is to the left of 'a', and the second line is to the left of 'f'. These lines represent the start and stop indices for the slice operation.

Left of index 1 *Left of index 6*

```
'bcdef'
```

game

PYOGGLE

Example: s = “Singaporean”

s[0:3] = “Sin” ← *Not* “Sing” !

s[:5:-2] + s[-1] = “neon”

s[0] + s[1:3] * 2 = “Singing”

s[7] + s[1: :-1] + s[-3] = “riSe” ← *Not allowed !*

s[2: :4] + “_” + s = “non-Singaporean”

Rules

- Can use + and *
- Bonus points for using step
- Bonus points for words longer than 7 letters

Dictionary

supercalifragilisticexpialidocious



supercalifragilisticexpialidocious

/ˌsuːpəkalɪfrədʒɪlɪstɪk ɪkspɪəlɪˈdeɪʃəs/ 

adjective informal

extraordinarily good; wonderful.

"the only word to characterize Kepler's discoveries was 'Supercalifragilisticexpialidocious'"



[Click to listen](#)

```
s='supercalifragilisticexpialidocious'  
0123456789111111111122222222223333  
012345678901234567890123
```

JupyterLab Shortcuts (1)

Two modes (know this first):

- **Command mode** → blue border → press `Esc`
- **Edit mode** → green border → press `Enter`

Run Cells (in Command mode)

- **Run & go to next:** `Shift + Enter`
- **Run & stay:** `Ctrl + Enter`
- **Run & insert below:** `Alt + Enter`

JupyterLab Command Mode Shortcuts

Add / Delete Cells

–(Command mode)

–Add above: A

–Add below: B

–Delete cell: D, D

Copy / Paste Cells

–(Command mode)

–Copy: C

–Paste below: V

–Paste above: Shift
+ V

JupyterLab Edit Mode Shortcuts

+ **Comment / Uncomment**

- Toggle comment:** `Ctrl + /`
- (works for multiple selected lines)

Indentation

- Indent:** `Tab`
- Dedent:** `Shift + Tab`

Survival Rule

If the shortcut doesn't work,
you're in the wrong mode.
Press `Esc` (cell actions) or
`Enter` (typing).